

Introduction to Software Architecture

Topu Newaj, CTO, BracIT Services.

Hands on 1

Software Architecture

Software architecture is the high-level blueprint that defines the structure of a software system. It is the foundation upon which system requirements, design, and performance criteria are built, encompassing everything from the behavior and interaction of individual components to the entire system's operational efficiency. A well-designed architecture is crucial for ensuring scalability, reliability, and flexibility to adapt to future requirements or changing technologies.

Software Architecture

Good software architecture considers the functional and non-functional requirements of a system, which include performance, security, scalability, and maintainability. It defines the boundaries within which developers work, guiding the design and implementation and helping reduce complexity by promoting the use of reusable components and standardized processes.

Understanding the Role of a Software Architect

A software architect is responsible for the technical vision and high-level structure of a software system. They bridge the gap between business goals and technical implementation, ensuring that all solutions align with both short-term and long-term objectives. Key responsibilities include:

- **Defining System Architecture:** Developing a structural blueprint that meets business and technical needs.
- **Evaluating Technologies and Frameworks:** Assessing tools, frameworks, and technologies for the system based on scalability, cost, and performance criteria.
- **Ensuring Compliance:** Making sure the architecture aligns with industry standards, security practices, and regulatory requirements.

Understanding the Role of a Software Architect

- Communication and Collaboration: Translating complex technical concepts into business terms for stakeholders, while guiding and mentoring the development team.
- Problem-Solving: Addressing issues that arise in both the design and implementation phases, especially in cases of unexpected requirements or technical limitations.
- A skilled architect balances technical acumen with leadership skills, ensuring the software system not only meets current demands but can also evolve over time.

Types of Software Architects

Application Architect

Focus: Specific applications or systems.

Responsibilities: Ensuring the architecture of a particular application aligns with company standards, selecting appropriate technologies, and optimizing the application's performance, maintainability, and scalability.

Key Skills: Application design patterns, coding, specific technology stacks.

Types of Software Architects

Solution Architect

Focus: Linking multiple applications and services within a specific project or solution.

Responsibilities: Creating and managing technical solutions that meet specific business requirements, designing integration points, and ensuring consistency across the components.

Key Skills: Integration techniques, middleware, knowledge of different subsystems.

Types of Software Architects

Enterprise Architect

Focus: Company-wide alignment of IT infrastructure and software systems.

Responsibilities: Defining strategic direction for technology, ensuring software aligns with business goals, setting architectural standards, and managing dependencies.

Key Skills: Business and technology alignment, standards development, governance frameworks.

Types of Software Architects

Technical Architect

Focus: Deep technical expertise in specific technologies.

Responsibilities: Addressing technical challenges, guiding development teams on technical issues, and ensuring best practices are followed.

Key Skills: Detailed technical knowledge, troubleshooting, technical mentoring.

Types of Software Architects

Infrastructure Architect

Focus: Underlying infrastructure (e.g., cloud, on-premise hardware, network).

Responsibilities: Designing the infrastructure, considering scalability, disaster recovery, performance, and cost-effectiveness.

Key Skills: Infrastructure provisioning, cloud services, scalability planning.

Data Architect

Focus: Data systems and models.

Responsibilities: Designing databases, data flow, storage systems, and ensuring data consistency, integrity, and security across systems.

Key Skills: Database modeling, data warehousing, ETL, security compliance.

Types of Software Architects

Security Architect

Focus: Security aspects of systems.

Responsibilities: Developing security models, policies, and standards, ensuring systems are resilient to threats and vulnerabilities.

Key Skills: Network security, cryptography, compliance frameworks (e.g., GDPR, HIPAA).

Career Progression

Foundational Knowledge (Junior Developer / Developer)

Skills: Strong programming skills, basic design patterns, fundamental understanding of software development life cycle (SDLC).

Activities: Work on feature development, bug fixes, and learn foundational aspects of software architecture, such as separation of concerns and solid code structure.

Intermediate Knowledge (Senior Developer)

Skills: Deeper understanding of design patterns, familiarity with system architecture concepts, experience with technical decision-making.

Activities: Begin to lead small projects or components, mentor junior developers, and get involved in architecture discussions and code reviews.

Career Progression

Broad Knowledge (Technical Lead)

Skills: Broader knowledge of different architectural patterns (e.g., microservices, event-driven architecture), technology stacks, and architectural decision-making.

Activities: Lead development teams, make higher-level design decisions, collaborate with architects to refine solutions, and begin focusing on non-functional requirements (scalability, maintainability).

Career Progression

Specialization (Architect)

Skills: Mastery of a specific architecture type (application, solution, etc.), high-level design, risk management, technology evaluation.

Activities: Design and guide implementation at the architectural level, work on ensuring best practices are followed, troubleshoot major architectural issues, and act as the main technical point of contact.

Career Progression

Advanced Knowledge (Senior Architect / Principal Architect)

Skills: In-depth expertise across multiple architectural domains, advanced strategic thinking, and understanding of business-technology alignment.

Activities: Shape and guide major projects, establish architectural standards, work on high-impact projects, and mentor other architects. Begin to participate in company-wide architectural governance and high-level strategic planning.

Career Progression

Strategic Knowledge (Enterprise Architect / Chief Architect)

Skills: Deep knowledge of the organization's business goals, cross-domain architecture, technology management, strong communication with executive stakeholders.

Activities: Establish company-wide architectural strategies, ensure alignment between IT and business goals, oversee architectural governance, and make high-level decisions impacting multiple teams and projects.

Hands on 2

Approaching Application Architecture

There are two fundamental approach towards Application architecture.

- Monolithic
- Distributed

The existence of monolithic and distributed architectures as two fundamental types is a natural outcome of balancing system complexity, scalability, resilience, development practices, and operational constraints.

Approaching Application Architecture

Monolithic architectures suit simpler applications or those with clear boundaries and limited scaling needs. They offer simplicity, ease of deployment, and performance benefits in smaller settings.

Distributed architectures cater to large, complex, or high-availability systems that require flexible scaling, fault tolerance, and adaptability to evolving technology. They introduce more complexity but offer better resource management and resilience to meet modern software demands.

Monolithic Architecture

Monolithic architecture is a traditional design where an application is built as a single, unified unit. Typically, this design organizes the software into layers (like presentation, business, and data access), all compiled and deployed together.

Key Characteristics

Unified Codebase: All components (UI, business logic, and data access) are tightly coupled and operate within the same process.

Single Deployment Unit: The application is built, tested, and deployed as a single package or file, meaning any changes require redeploying the entire application.

Tightly Coupled Components: The interdependence of components within the same codebase allows easy access to shared memory and resources, but it can hinder flexibility.

Pros

Simple Development and Testing: Since everything resides in a single codebase, development and testing processes are straightforward.

Performance: Communication between components is in-process, making it faster as there is no network overhead.

Easy Debugging and Tracing: With all components in one place, issues are easier to debug and resolve.

Easier Deployment: A single deployable unit simplifies deployment processes, often reducing the need for sophisticated DevOps setups.

Cons

Limited Scalability: Scaling a monolithic app typically means deploying multiple copies of the entire application, which can be resource-intensive.

Challenging to Maintain and Update: As the application grows, managing and updating code becomes complex, with risks of side effects in other parts of the codebase.

Reduced Flexibility: Adapting individual components without affecting others is difficult, leading to potential downtime or extensive regression testing.

Technology Lock-In: Since everything runs in a single process, changing the technology stack for a specific part of the application is challenging.

When to Use

Smaller applications or those with relatively simple requirements.

Projects that don't require high scalability or are not expected to grow extensively in complexity.

Environments where simplicity and fast development cycles are prioritized.

Monolithic Architecture Types

- Layered (N-tier) Architecture
- Modular Monolithic Architecture
- Microkernel Architecture

Layered (N-tier) Architecture

The layered architecture, also known as N-tier architecture, organizes the code into distinct layers, such as presentation, business, and data access. Each layer has a specific responsibility, providing a clear separation of concerns within a single application.

Structure:

- Presentation Layer: Handles the user interface (UI) and user interaction.
- Business Logic Layer: Processes and manages core business logic, rules, and workflows.
- Data Access Layer: Manages database connections and data storage interactions.

Layered (N-tier) Architecture

Benefits:

Separation of Concerns: Different aspects of the application are clearly separated, making it easier to develop and test.

Layering within a Monolith: While the application is a single deployment, it still benefits from internal layering.

Use Cases:

Best suited for small to mid-sized applications where scaling out as a single unit is feasible.

Common in traditional enterprise applications, like order management and inventory systems, where layered design helps manage complexity within a monolith.

Modular Monolithic Architecture

A modular monolithic architecture organizes the codebase into separate modules within the same application. While the entire application is deployed as a single unit, the code is structured into independent modules that interact with each other in a well-defined way.

Structure:

- Modules are organized around distinct features or services (e.g., billing, inventory, and customer management).
- Each module handles a specific domain but shares the same codebase and runtime environment.

Modular Monolithic Architecture

Benefits:

Better Manageability: Modules can be developed and tested independently while still being part of the same application.

Logical Separation: Modules provide a sense of isolation without the complexity of a distributed system.

Use Cases:

Suitable for larger applications where different functionalities are closely related but benefit from internal separation.

Common in enterprise systems like ERPs that have multiple interconnected business functions within a single deployable unit.

Microkernel Architecture

In a microkernel or plug-in architecture, the core system provides essential functions, while additional features are added through plug-ins or modules. This allows for extensibility within a monolithic structure, where the core remains intact as features are added.

Structure:

Core System (Kernel): The core provides essential services and a stable interface.

Plug-ins: Additional functionality is added via plug-ins, which extend the core without altering it.

Microkernel Architecture

Benefits:

Flexibility and Extensibility: New features can be added without modifying the core, allowing for customization.

Isolated Development: Plug-ins can be developed independently and integrated as needed.

Use Cases:

Ideal for applications requiring core functionality with optional add-ons, like CMS (e.g., WordPress) or IDEs (e.g., Eclipse).

Useful for systems where the core functionality remains stable, but extensibility is a priority.